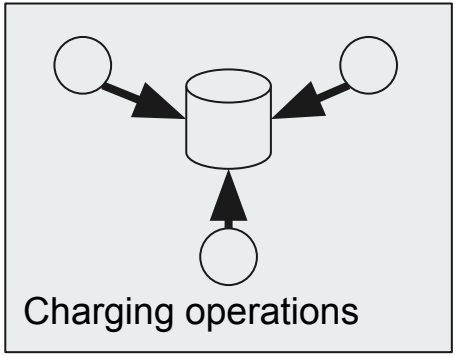
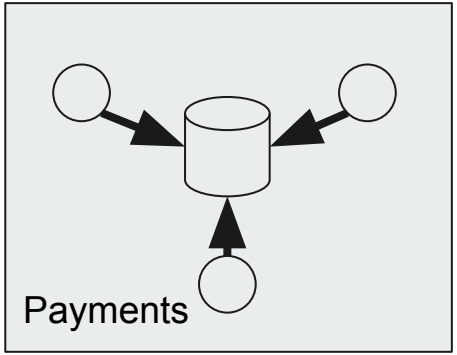
Eat your own logfood

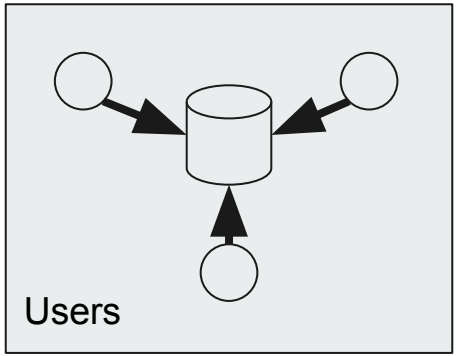
Dogfooding:

the idea that someone would use the products they were making became known as "eating your own dog food." [1]

Background: Rough Org Structure



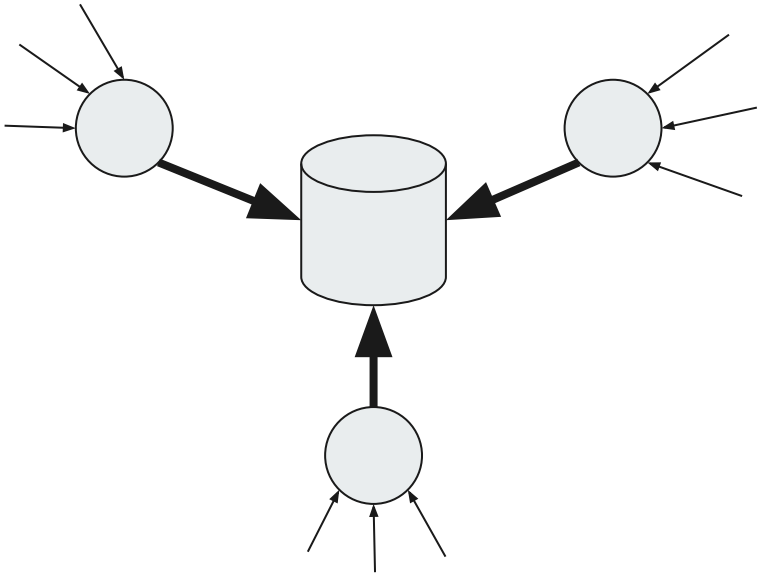




Problems We Encountered

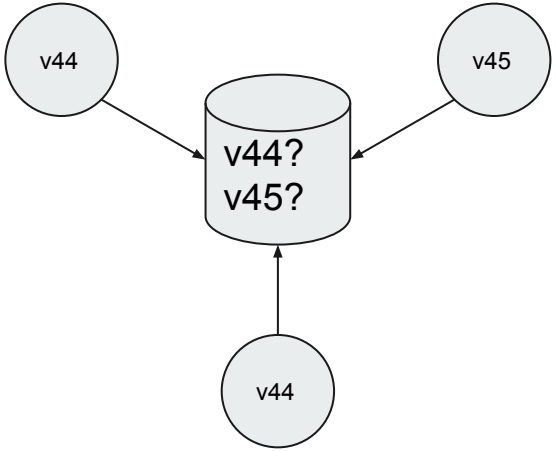
Existing Problem #1: Central MariaDB

- Single point of failure
- Payment Service's performance bottleneck

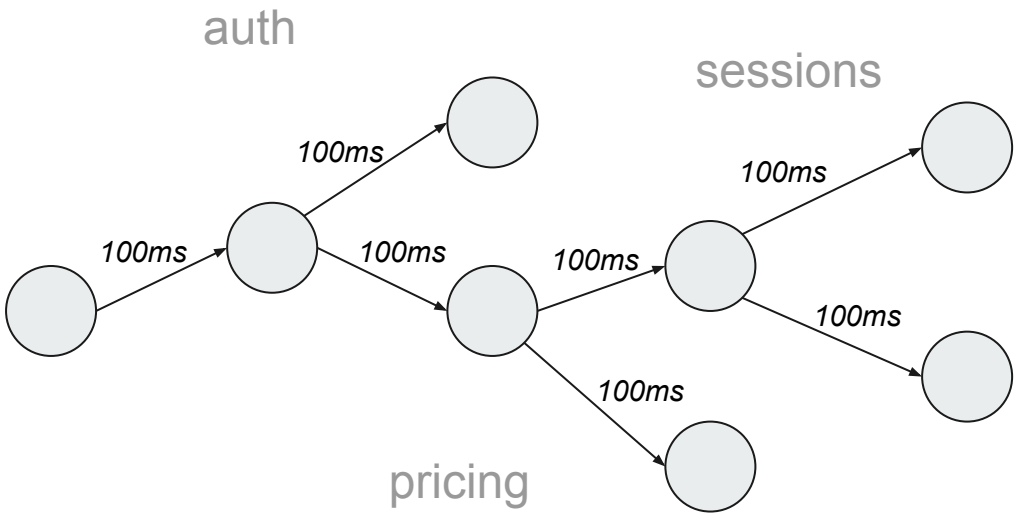


Existing Problem #2: Flyway Migrations

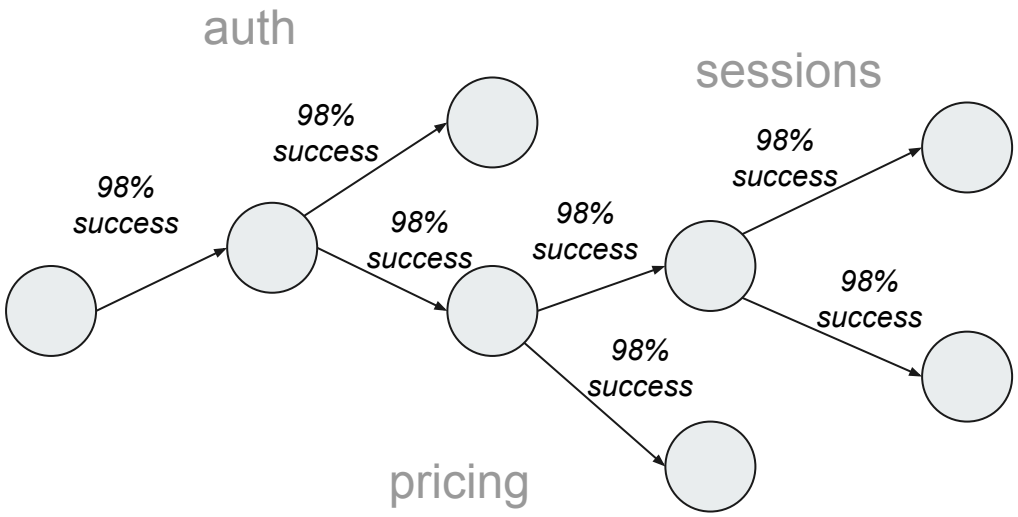
- Destructively updates your data
- Not testable
- Requires downtime
- Not flexible
- Need to delete or invent history
- Doesn't support 'rolling replacements' of payments service instances



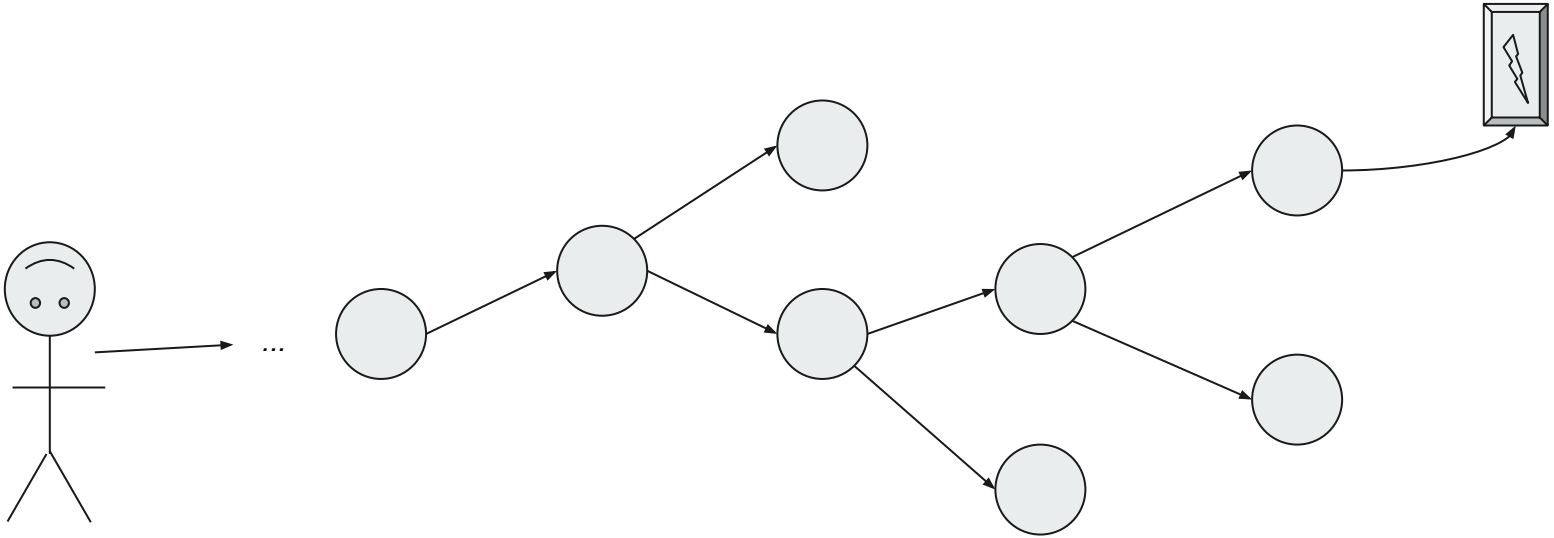
Existing Problem #3: High latency services



Existing Problem #4: Flaky services



Existing Problem #5: Synchronous Services



Existing Problem #6: Inflexibility of SQL

Sessions

SessionId	UserId	CustomerKeyId	
SessionId(123)	UserId(456)	CustomerKeyId(456)	
SessionId(234)	{Anonymous}	{No Customer Key}	Phone No (555-1234)

Existing Problem #6: Inflexibility of SQL

SessionId	UserId	CustomerKeyId	
SessionId(123)	UserId(456)	CustomerKeyId(456)	
SessionId(234)	{Anonymous}	{No Customer Key}	Phone No (555-1234)

```
{ "type":          "UserSession”
, "SessionId":     "sessionId"
, "UserId":        "userId"
, "CustomerKeyId": "customerKeyId"
}
```

```
{ "type":        "AnonSession"
, "SessionId":   "sessionId"
, "PhoneNumber": "phoneNumber"
}
```

Existing Problem #7: Unusable features

“We built the history service.”

“Please don’t use the history service; you’ll stop people from charging”

Missing Features

Missing Feature #1: Reliable Auditing/Logging

- **We had so. much. logging.**
- Cannot do anything with the data
- Cannot ask simple questions - need to do forensic investigations
- Logs don't always match reality
- Spent a really long time scrolling through textboxes on Amazon

- **Yet we only had "developer logs", not "real audit logs".**

CPO:

```
session.start(request.startTime, connector, request.authorization)
        .onSuccess(event -> eventSource.fireEvent(new SessionStartGrantedEvent(request.authorization.user,
                                                  session.id)))
        .onSuccess(event -> sessionEventRepository.saveEvent(session.id,
                                 event))
        .onSuccess(event -> chargeServiceClient.notifyChargeStarted(session.sessionReference,
                                         session.id,
                                         chargePoint.owner,
                                         event.startTime,
                                         connector))
        .onFailure(failure -> eventSource.fireEvent(new SessionStartDeniedEvent(request.authorization.user,
                                                  session.id,
                                                  SessionStartDeniedEvent.Reason.BUSINESS_RULE_VIOLATION,
                                                  failureToDetails(failure))))
        .onFailure(failure -> chargeServiceClient.notifyChargeStartFailed(session.sessionReference,
                                         failureToDetails(failure))));
```

EMP:

```
getSession(request.sessionId)
        .flatMap(session -> session.registerStarted(request.sessionReference,
                                 request.chargePointOwner,
                                 request.startTime,
                                 request.connectorReference).mapFailure(Failure.class::cast)
        .onSuccess(event -> eventSource.fireEvent(new SessionStartConfirmationGranted(actor, request.sessionId)))
        .onSuccess(event -> sessionEventRepository.saveEvent(request.sessionId, event))
        .onSuccess(event -> sessionTimeoutManager.cancelStartTimeoutJob(request.sessionId))
        .onFailure(failure -> eventSource.fireEvent(new SessionStartConfirmationDenied(actor,
                                                  request.sessionId,
                                                  SessionStartConfirmationDenied.Reason.BUSINESS_RULE_VIOLATION,
                                                  failureToDetails(failure)))));
```

Logging code

CPO:

```
session.start(request.startTime, connector, request.authorization)
        .onSuccess(event -> eventSource.fireEvent(new SessionStartGrantedEvent(request.authorization.user,
                                                  session.id)))
        .onSuccess(event -> sessionEventRepository.saveEvent(session.id,
                                    event))
        .onSuccess(event -> chargeServiceClient.notifyChargeStarted(session.sessionReference,
                                          session.id,
                                          chargePoint.owner,
                                          event.startTime,
                                          connector))
        .onFailure(failure -> eventSource.fireEvent(new SessionStartDeniedEvent(request.authorization.user,
                                                  session.id,
                                                  SessionStartDeniedEvent.Reason.BUSINESS_RULE_VIOL
                                                  failureToDetails(failure))))
        .onFailure(failure -> chargeServiceClient.notifyChargeStartFailed(session.sessionReference,
                                          failureToDetails(failure))));
```

EMP:

```
getSession(request.sessionId)
        .flatMap(session -> session.registerStarted(request.sessionReference,
                                    request.chargePointOwner,
                                    request.startTime,
                                    request.connectorReference).mapFailure(Failure.class::cast)
        .onSuccess(event -> eventSource.fireEvent(new SessionStartConfirmationGranted(actor, request.sessio
        .onSuccess(event -> sessionEventRepository.saveEvent(request.sessionId, event))
        .onSuccess(event -> sessionTimeoutManager.cancelStartTimeoutJob(request.sessionId))
        .onFailure(failure -> eventSource.fireEvent(new SessionStartConfirmationDenied(actor,
                                                  request.sessionId,
                                                  SessionStartConfirmationDenied.Reason.BUSINESS_RULE
                                                  failureToDetails(failure)))));
```

Missing Feature #2: Analytics Data

- Business types and data geeks need our data
- “Data-lake” is the buzzword of the day here

How to fix?

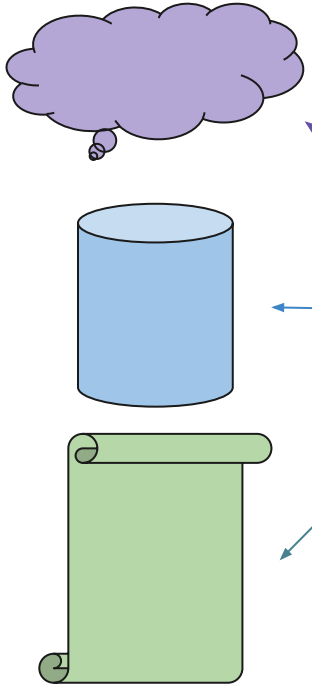
By the way, logging sucks!

- It is unclear where/when/how to do it
- It's extra lines of code
- It is as complicated as any feature, yet:
    - People assume it *just works*
    - It could throw and exception and break your system
    - It could do blocking IO and slow down your system
    - It is almost never tested. Find me a test in your test suite that asserts that a log line was generated!

How is logging generally used?

```
// Notify video service that user disconnected.
videoService.userDisconnected(userId);
log.info("user " + userId + " disconnected from video service.");
```

How is logging generally used?



```
// Notify video service that user disconnected.
videoService.userDisconnected(userId);
log.info("user " + userId + " disconnected from video service.");
```

Thought experiment

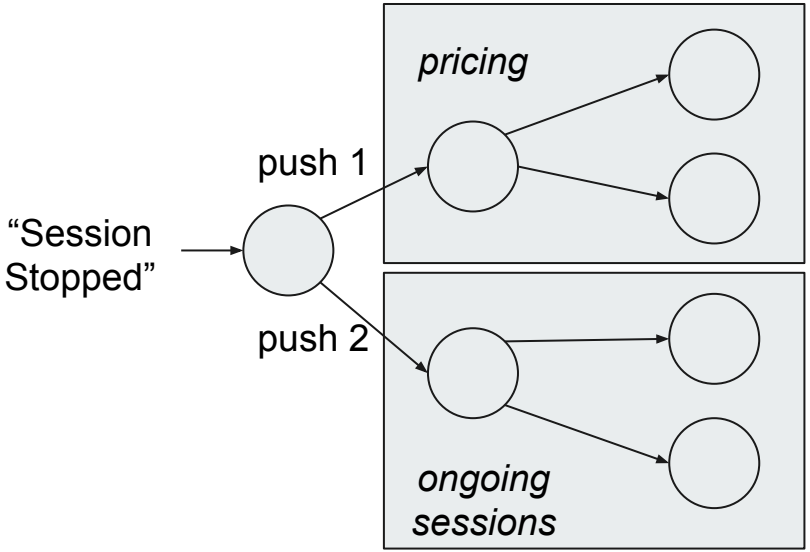
Why can't we just do the following?

~~// Notify video service that user disconnected.~~

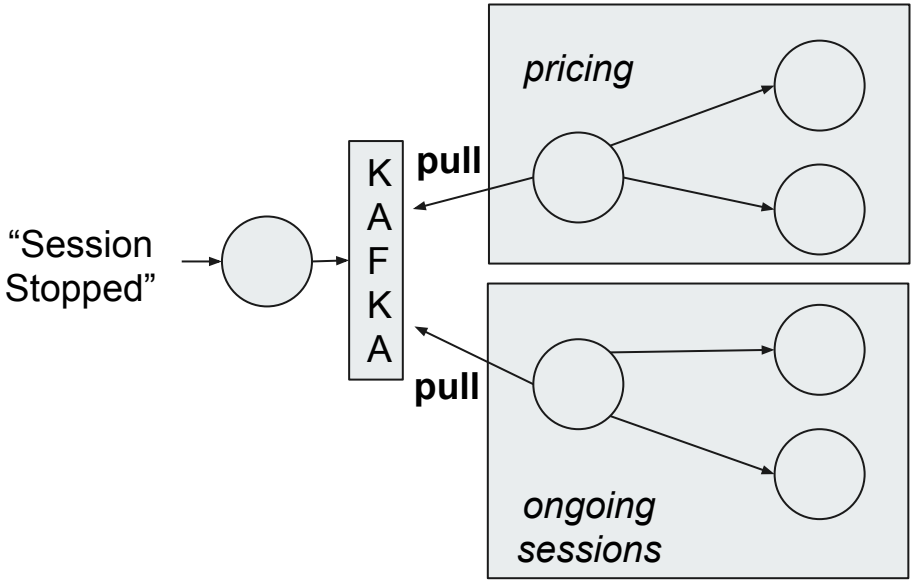
~~videoService.userDisconnected(userId);~~

```
log.info("user " + userId + " disconnected from video service.");
```

Eat your own log food!



vs.



Tip: Log *real-world* events

- Log *what actually happened*.
- Log things that other computers can do something about.

The consumer(s) of the logs can decide the meaning(s).

- {Charge Started} instead of {Starting Charge}
- {Customer Paid $3.50} instead of {Calculating Fee}
- {} instead of {Timing out user}

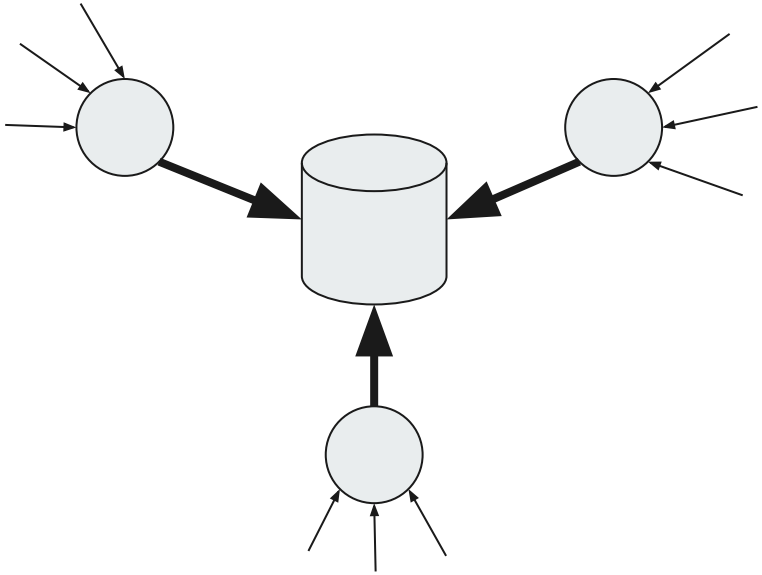
Reliability win: Independent Failures

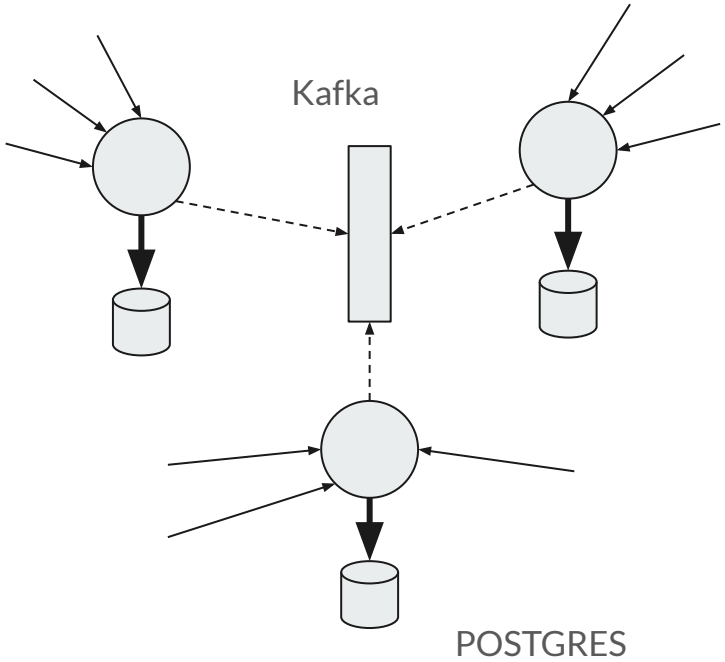
A big failure in the payments service *used to* take out all functionality.

Now History Service can go down, and only the history view breaks - everything else keeps running.

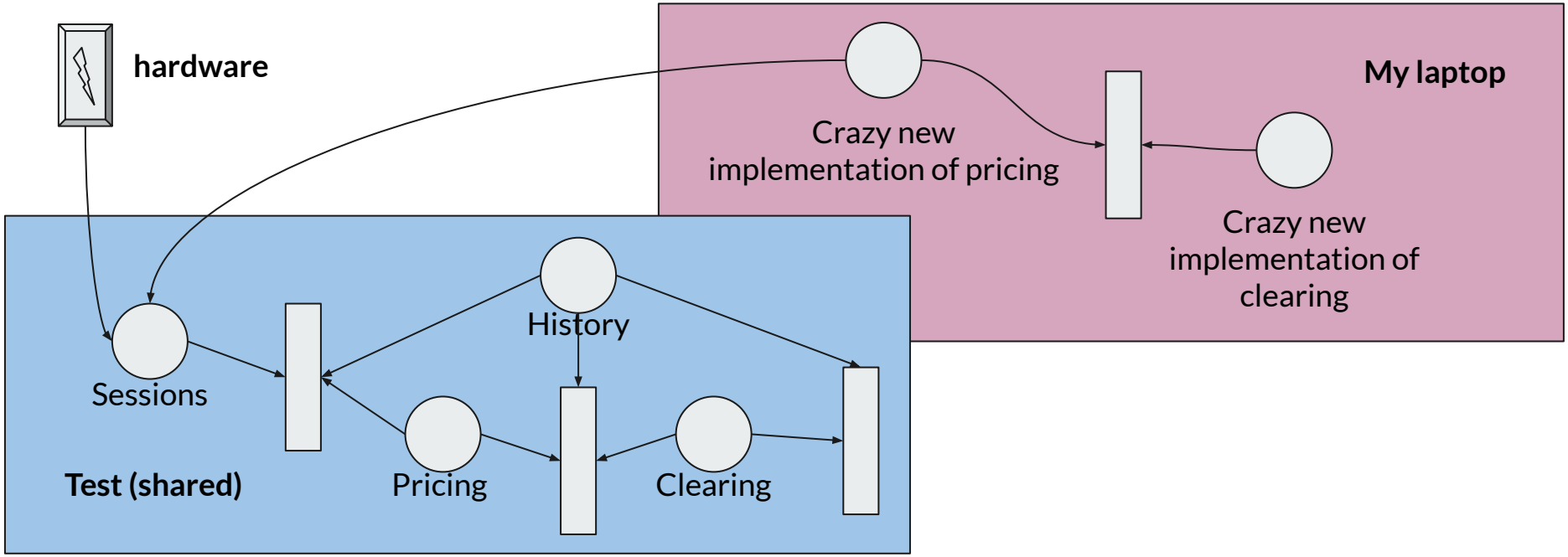
Clearing Service can go down, and we don't lose any money, because it no longer has to collect money at the same time the charging happens.

Scalability win

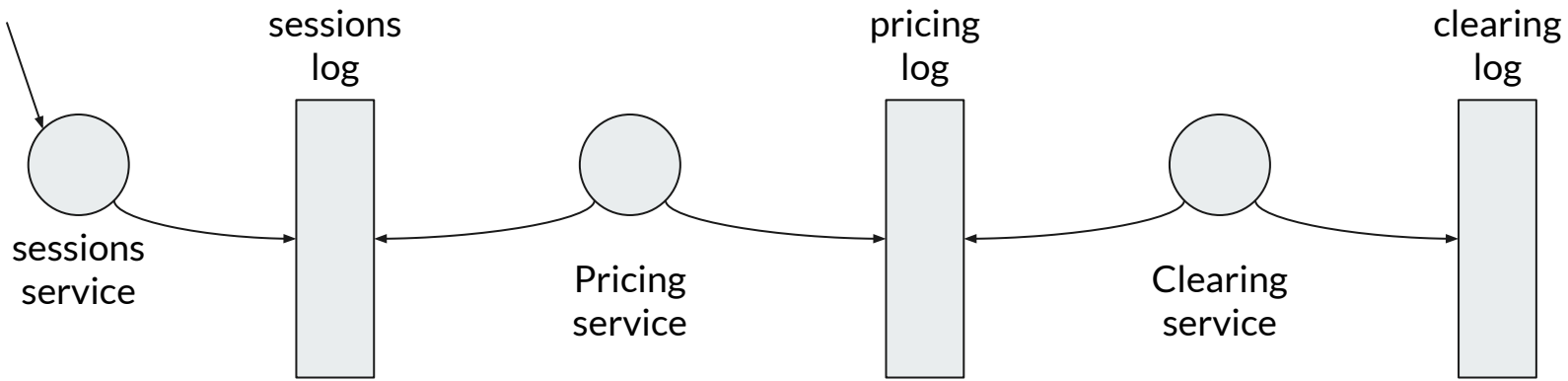




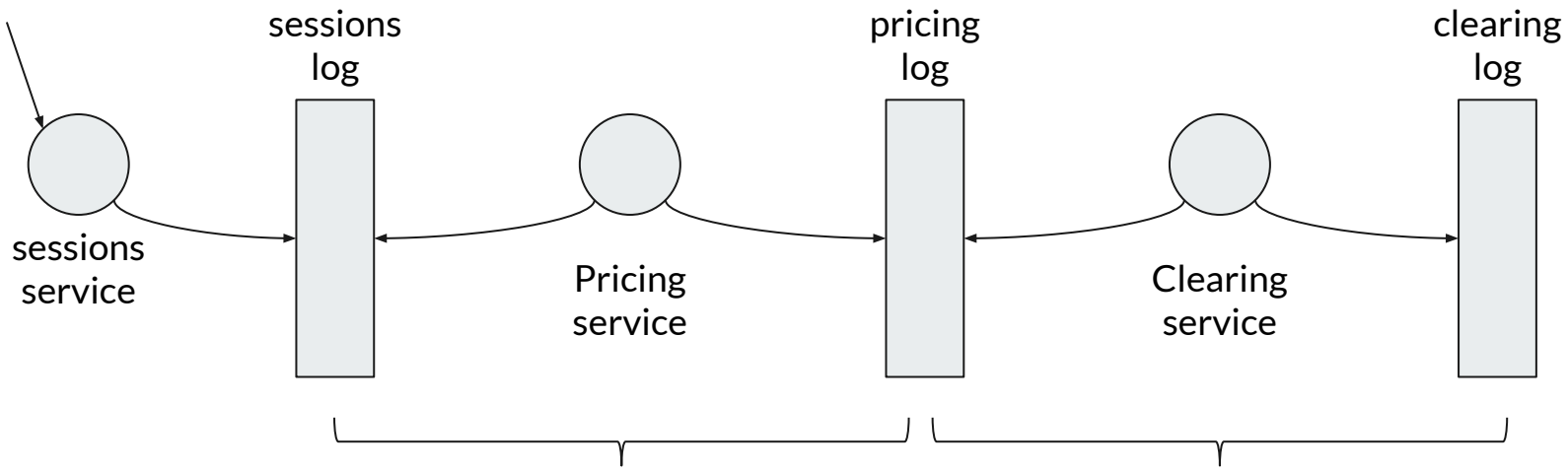
Developer velocity win



Correctness win: Independent Verification



Correctness win: Independent Verification



"Did every session create a debit on a customer?"

"Has every customer credited the same amount as their debits?"

Why Kafka *specifically*?

It is **not about message passing**. Heaps of libraries do that.

It is about getting **multiple computers** to agree on the **contents and ordering of an append-only log.**

So that other systems can change themselves in response to the events on that log.

What’s it called when your system changes itself in response to a stream of inputs?

Problems

Summary problems with Kafka approach

Problem #1 - Harder to ‘correct’ prod data

- You can’t just change someone’s bank balance

- **But**, this leads to more auditable actions!
  - You keep the bad transactions in the system and follow them up with ‘fixer’ transactions.
  - You have proof of what went wrong and how you fixed it (i.e. **an audit trail**)

Problem #2 - Harder to ‘migrate’

- You can’t (easily) update old data to add/remove fields.
  - In theory this makes sense. If you didn’t record a timestamp at the time, how can you go back and add one later?
  - Maybe you need to be explicit with your versioning. All of our domain objects ended up being appended with V1.

- **But**, the kafka topics aren’t the only views of the data. You can nuke the downstream sql databases and recreate them with new schemas.

Problem #3 - Infeasible to Replay Whole World

- It will take longer and longer to re-play every message.
- Need one or more solutions of dealing with this.
  - Maybe start a new log every month/year?
  - Figure out when data can be retired/archived?
  - Figure out ‘safe start points’ where a service can start from?

Problem #4 - The boundary problem

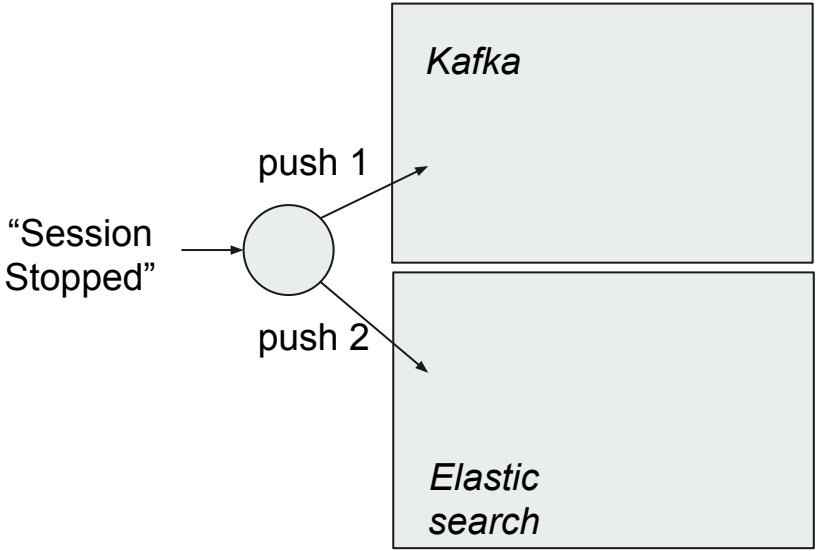
- Not every service uses the log-everything approach.
- Once you put some logic into a permanent log, you will want to add all the other logic too! It's hard to find the right boundary to cut things off.

Problem #5 - Other teams

People will try to reintroduce the problems you removed.

Other teams will want to:

- Push into Kafka
- but also push into ElasticSearch
- NO!



Problem #6 - Crossover

- We effectively rewrote the whole system
- We ran it concurrently with the old system

Problem #7 - GDPR

GDPR and Kafka are completely opposing forces:
The benefit of Kafka is it reliability **keeps every piece of your history**
The point of GDPR is to allow people to **remove themselves from your history**

You will need some clever tricks, e.g.:

- Store users another service, and refer to them via external/virtual ids. Then the user service can just sever the link between virtual and real IDs.
- Some organisations encrypt user records and throw away the key when the user requests deletion.
- Backups! If you have backups you might accidentally recreate sensitive user data (not Kafka specific). Keep a delete list?
- Financial requirements often beat GDPR requirements

Summary: Problems we addressed

- Made it easier to know what to log and why
- No more centralised SQL server performance bottleneck
- Kafka is NoSQL -> no schema -> no Flyway migrations
- Services independently pulling the same data -> Less chance of inconsistent state
- Developer velocity: updating code on local laptop while reacting to test environment data (less need to commit/review/merge/test.)
- Less flakiness since the user no longer has to interact with the whole system end-to-end (they are done once their message is committed to the first kafka log)
- Got an “audit log” “for free”
- Dodged the integration with analytics team (we don’t need to choose what we publish to them - we publish everything to us. The analytics team can choose what they want to consume)
- Real-time financial balancing
- Independent failures. Pricing/clearing services can go down and catch up later (users can still charge their cars)